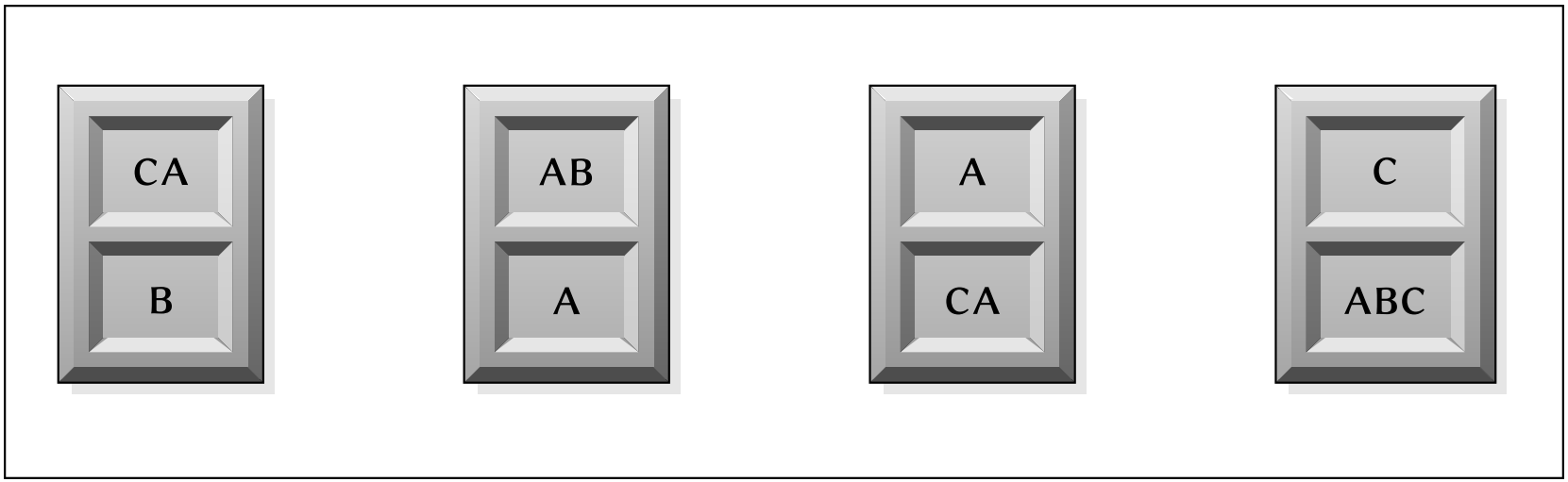
The Post Correspondence Problem

This week we'll be showing that the Post Correspondence Problem (the PCP) is undecidable.

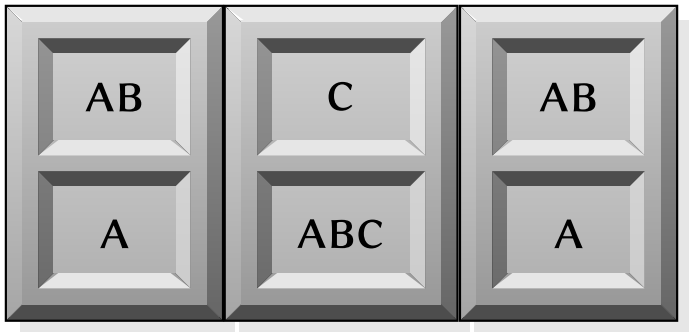
You've seen this problem already in tutorials, but let's review it here as well.

To start off, I give you a set of tiles:



The Post Correspondence Problem (2)

We can take copies of these tiles and line them up:



> *Notice how we use tile number two twice in this sequence. We can use each tile type as often as we want.*

When we take a sequence of tiles like the one above, we can read off two strings: one from the top and one from the bottom:

Top string:	ABCAB
Bottom String:	AABCA

In this case the top and the bottom string are different.

The Post Correspondence Problem (3)

The question is this: If I give you a set of tiles such as the one above, can you find a non-empty sequence of tiles such that the top and bottom strings are the same?

> **The Post Correspondence Problem (the PCP):**
>
> **Input**: A finite set of tiles.
> **Question**: Can we find a non-empty sequence of tiles such that the top and bottom strings read from that sequence are equal?

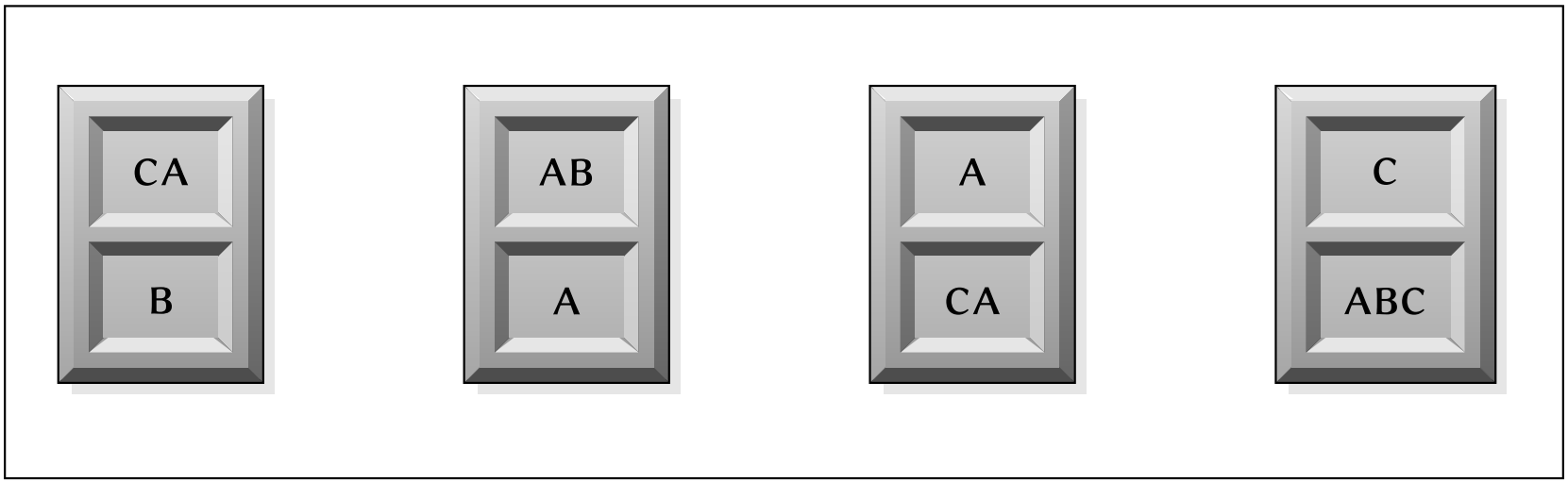
As always, the string encodings of yes-instances for this problem form a language. We'll refer to this language as the PCP, as well.

This is a simple problem, so it's probably surprising to find that it's undecidable! We'll prove this soon.

Why is the PCP undecidable?

Idea: Use a mapping reduction — we'll emulate a TM operation with a carefully chosen set of tiles.

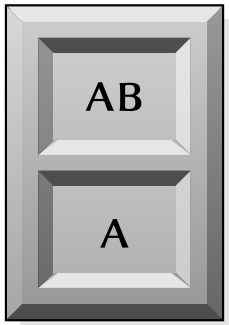
Recall our example — when we had the tile list



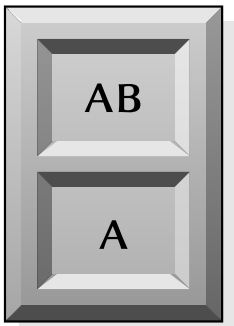
- there was only one possible starting tile, and
- once we chose that starting tile, the rest of the sequence could be "grown" be predicting the one possible next tile.

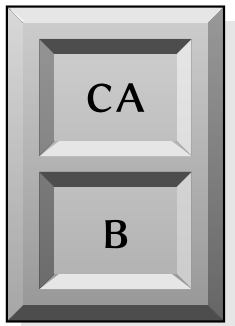
PCP Matching

That is, the only tile we can start with is



since it is the only tile with the same character starting the top and bottom string. But then, we have to follow it with





Since the next character of the bottom string has to be a B. And we can continue on from there.

General Mapping Reductions

Up until now most of our reductions have been to TM-related languages. Here's where we really start to move into non-TM formlisms.

Later we'll be mapping into:

- Graph problems
- Timetable scheduling
- Games and puzzles
- and more...

We'll need to figure out how to "program" in these formalisms.

> *If we play with a formlism and find a way of forcing it to act in a certain way, that's often a good start.*

Our forced choice tile is a key to programming in the PCP.

A First Attempt

If we're careful about how we build the tiles, we can force the choice of the first tile in the match, the second, and so on.

OK, so what sort of strings do we want to generate?

Well, we're emulating a TM. So let's start with the TM/input pair $\langle M, w \rangle$.

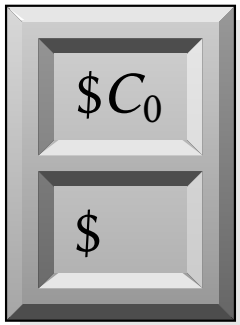
Let's try generating the configuration history:

$$\begin{array}{lcl} C_0 & = & q_0 w \\ C_1 & & \\ C_2 & & \\ & \vdots & \end{array}$$

These configurations are strings — and we build our PCP tiles using strings. So we can use the configurations to build PCP *tiles.*

A First Attempt (2)

Let's build a set of PCP tiles like so: our first tile will be

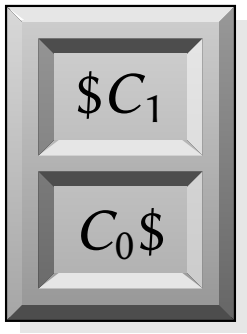


> *As usual, we're using '$' to denote a character that doesn't occur in any of the configurations.*

This will be like our first character from our previous example — as long as we don't let any other tile have top and bottom strings with the same initial character, this will have to be the starting tile.

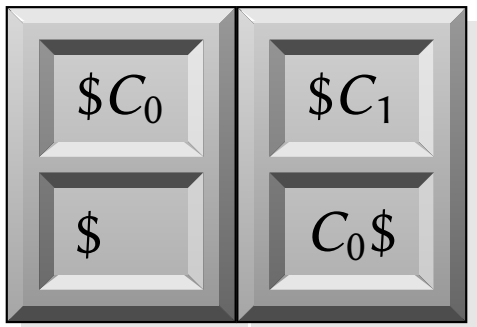
A First Attempt (3)

Now, we can extend the tile set by calculating the next configuration and placing it on a tile like so:



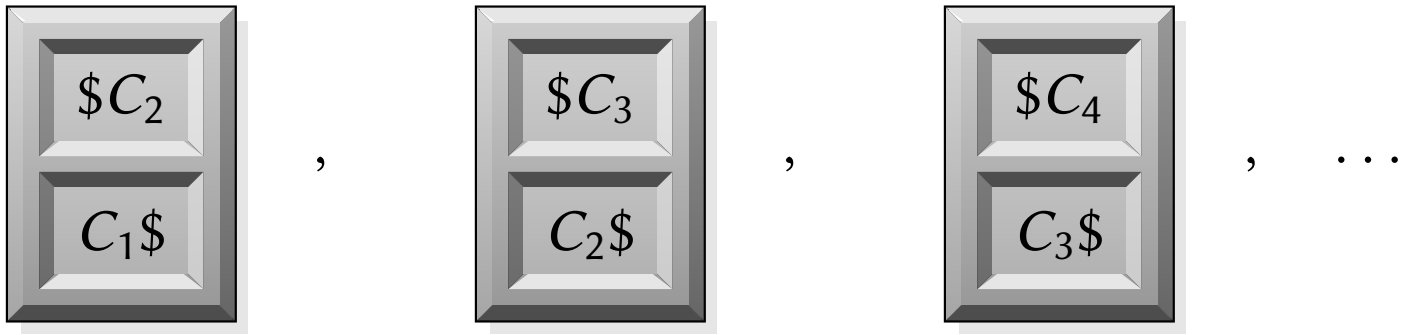
We can do this — these are just strings, after all, and we can figure out what they will be by looking at M and w.

If we try to use these tiles to build a sequence like we did last time, we get

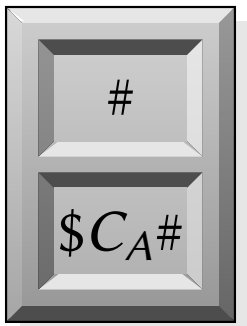


A First Attempt (4)

Let's just repeat the process:



If we reach an accepting configuration, we just add in a finishing tile



and we're done! *Or are we?*

A First Attempt — Why it Doesn't Work

Problem: We need one tile for every step the TM takes. If it loops, we'd need an infinite number of tiles. But a PCP instance should have a finite number of tiles.

The mapping reduction can't take an infinite amount of time, either.

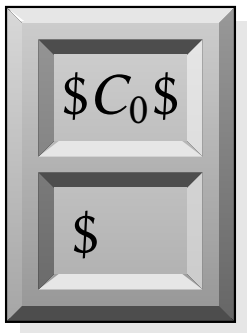
On the other hand, the basic idea is sound:

- There are no problems with using configuration strings in the tiles.
- Figuring out the individual configurations wasn't that bad.
- Using the top/bottom matching to force the next choice worked perfectly.

So let's try again.

A Second Attempt

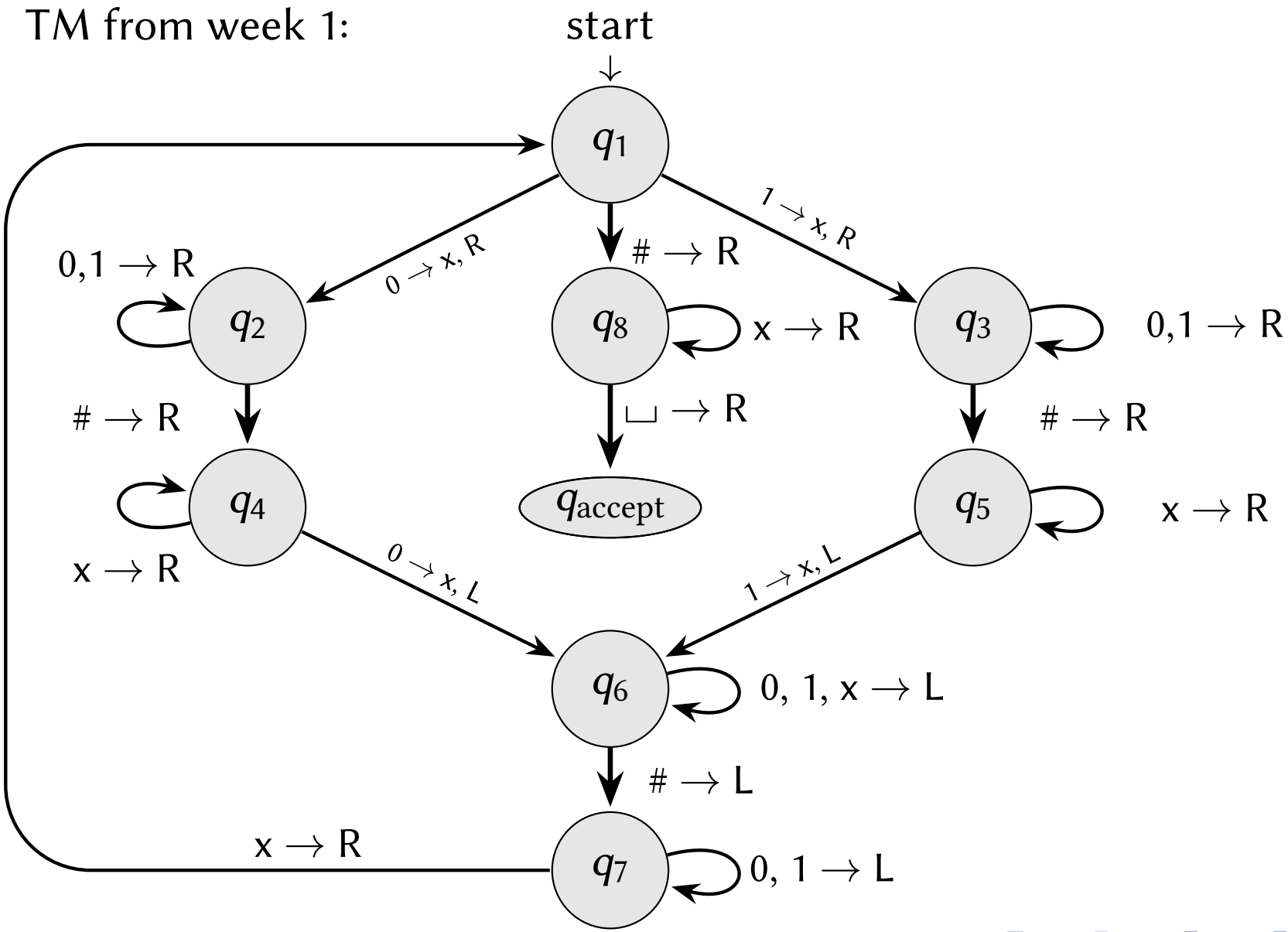
Idea: We'll follow the same basic idea, and even use the same starting tile:



The rest of the tiles will be built character-by-character.

A Second Attempt: An Example TM

TM from week 1:



A Second Attempt: An Example TM (2)

Given the input $w = 010\#011$, the first few configurations were:

$$
\begin{aligned}
C_0 &= q_1 010\#011\sqcup \\
C_1 &= \mathrm{x} q_2 10\#011\sqcup \\
C_2 &= \mathrm{x}1 q_2 0\#011\sqcup \\
C_3 &= \mathrm{x}10 q_2\#011\sqcup \\
C_4 &= \mathrm{x}10\# q_4 011\sqcup \\
&\vdots
\end{aligned}
$$

Now, suppose we'd set up our tiles to generate these configurations (as in the last example). So our strings would be:

$$
\begin{aligned}
\text{Top:} \quad & \$C_0\$C_1\$C_2\$C_3\$C_4\$ \\
\text{Bottom:} \quad & \$C_0\$C_1\$C_2\$C_3\$
\end{aligned}
$$

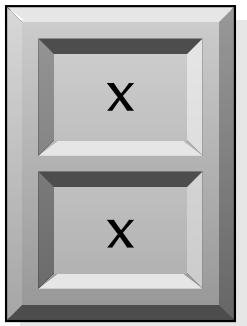
A Second Attempt: An Example TM (3)

Now, we want to build C_5. The first character will be x.

Do we need to know what the head is going to do in the next step to figure this out? Why not?

> No. None of the possible steps moves the head enough to change this character.

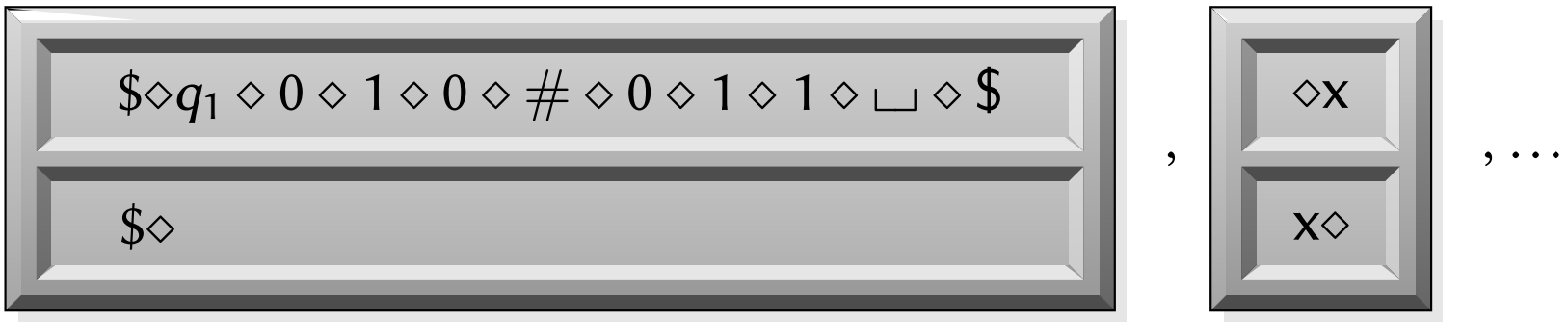
Idea: The next character will be something like



A Problem

Problem: This tile on its own will be a matching sequence! There's nothing to force us to start with our original initial tile!

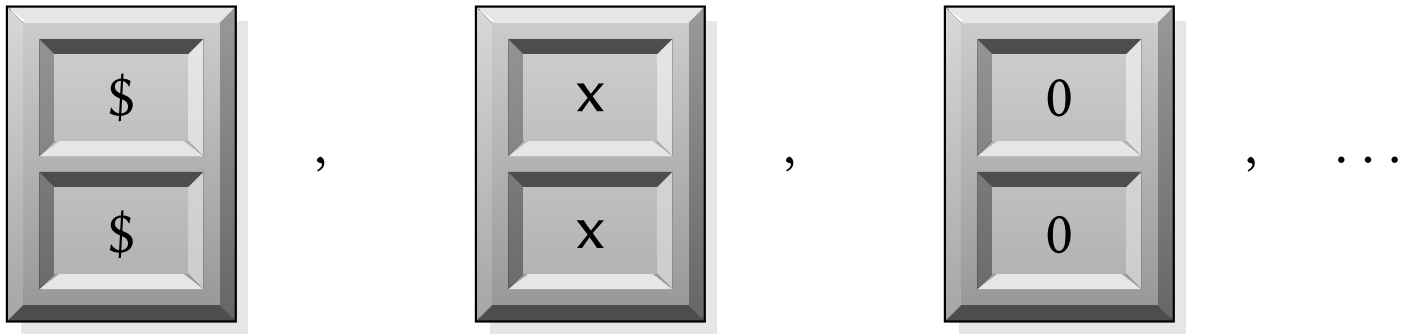
Solution: A new special character $\diamond$ interleaving between every other character:



This'll work, but will get really messy to write. So the $\diamond$ will be on all of the tiles, but we won't write it unless we need to.

A Second Attempt: An Example TM (4)

With that dealt with, we can put in a tile for every non-state character:



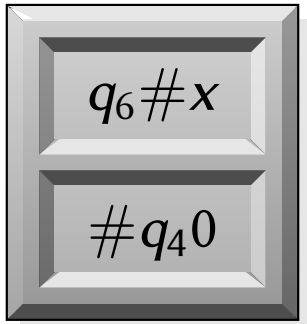
Most of the characters in C_5 are the same as in C_4, so these tiles will let us copy those characters over without any problem.

The only characters we need to worry about are the state characters and the characters that are changed as we move from C_4 to C_5.

> *But you know from assignment 1 that there's a limit to how many characters can change between configurations — in fact, that any change has to occur in a small window around the TM head.*

A Second Attempt: Finishing the Example

Idea: We can build a tile that shows just this window around the TM head. For example, the tile



would let us encode the difference between C_4 and C_5 in our example.

> *The important thing is that the window size is bounded, so the number of tiles we need to use to encode these differences is also bounded.*

So if we put all of these tiles together, we can extend our tile matching one character at a time in a way that corresponds exactly to the TM operation!

> *If we put in a set of tiles around the accept and reject states q_A and q_R that let us close off our matching, we'll have a matching sequence of tiles iff M halts on w.*

The PCP is Undecidable: an Overview

Here are a few high-level takeaways to remember:

- We can encode TM operation into problems not involving TMs to get a wide range of undecidability results.
- These encodings are possible because of the local nature of head movement on the tape in TMs.
- Because of these reductions, we can effectively run TMs in many different media: if there's a problem that we can do using the PCP, we can also solve it using a TM, and so on.
- We'll see a similar TM embedding later in the course when we look at Cook's theorem: the same basic idea works (with some modifications) even if make M a nondeterministic TM, or if we say that it has to finish in a particular number of steps.

Why the Undecidability of the PCP is Important

So we know that the PCP is undecidable. Now, let's show why we care about the PCP in the first place. After all, the PCP doesn't show up more often than TMs do.

But, it is a very good jumping-off point for many mapping reductions that don't use TMs.

> *How many of you remember the CFGs from CSCB36?*

Remember that a *CFG* is a tuple (V, Σ, P, S), where:

- A set V of non-terminal elements.
- The set Σ is the set of terminal elements — or our alphabet. Note that no terminal is a non-terminal: $V \cap \Sigma = \varnothing$.
- A set P of production rules.
- A special start nonterminal, usually denoted S.

CFGs: An Example

Consider the CFG

$$S \to \varepsilon \mid 0 \mid 1 \mid 0S0 \mid 1S1$$

This grammar generates all of the palindromes over the alphabet $\Sigma = \{0, 1\}$: The set V of non-terminals would be $\{S\}$, the production rules P would be the five right-hand side options above, and the start symbol would be S.

E.G, we could use this grammar to generate the strings 001100 and 01010 as follows:

$$S \Rightarrow 0S0 \Rightarrow 00S00 \Rightarrow 001S100 \Rightarrow 001\varepsilon100 = 001100$$

and

$$S \Rightarrow 0S0 \Rightarrow 01S10 \Rightarrow 01010$$

An Undecidable CFG Problem

What we'll do today is show that there are undecidable problems relating to CFGs. In particular, the following language is undecidable:

$$\text{CFG-Intersection} = \left\{ \langle G_1, G_2 \rangle \;\middle|\; \begin{array}{l} G_1 \text{ and } G_2 \text{ are CFGs} \\ \text{such that } L(G_1) \cap L(G_2) \neq \varnothing \end{array} \right\}$$

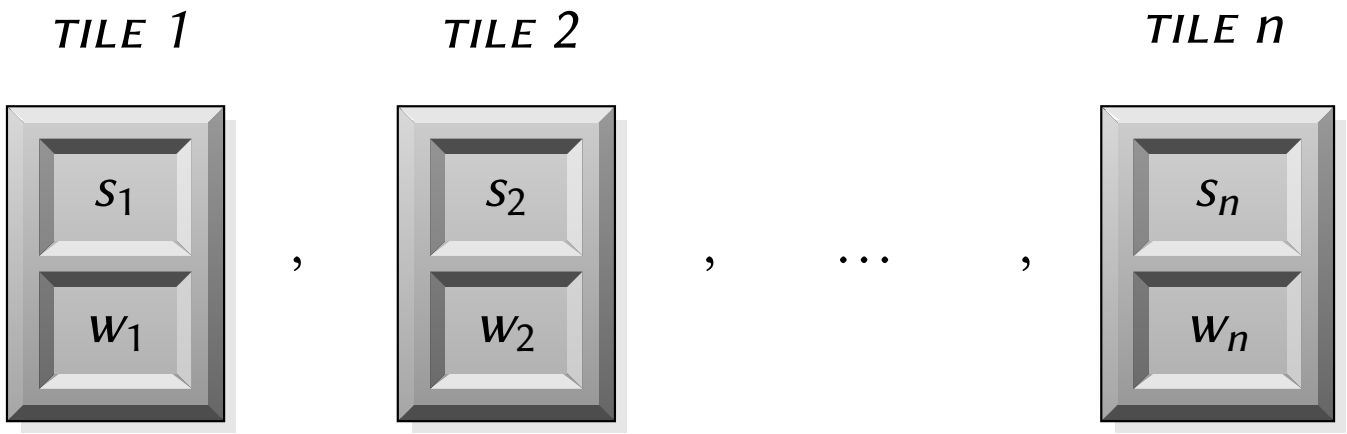
There's no general algorithm that can tell us, given an input of two CFGs, if there's any string can be generated by both grammars!

But how can we prove this?

- Generally, we reduce from HALT or $\overline{\text{HALT}}$, but that's difficult here.
- But if $A \leqslant_m B$ and $B \leqslant_m C$, then $A \leqslant_m C$!
- So since we know that the PCP is undecidable, we can reduce from that instead.

Reducing from the PCP

Here's the idea: Suppose we have a PCP instance:



Note that s_i is the string on the top of the i^{th} tile, and w_i is the string on the bottom.

Reducing from the PCP (2)

Idea: Try writing two CFGs:

- The first will generate the top strings from tile sequences.
- The second will generate the bottom strings from tile sequences.
- Both will remember which tiles are used.
- Since our grammars will generate the top and bottom strings, a match in the PCP instance will correspond to a string in both languages.

In these grammars, let t_1, t_2, t_n be special characters not present in any of the w_i or s_i. They'll represent which tiles are used.

Reducing from the PCP (3)

With that in mind, here are our two grammars:

$$\begin{aligned} G_1: \quad & S_1 \to s_1 A_1 t_1 \mid s_2 A_1 t_2 \mid \ldots \mid s_n A_1 t_n \\ & A_1 \to \varepsilon \mid s_1 A_1 t_1 \mid s_2 A_1 t_2 \mid \ldots \mid s_n A_1 t_n \end{aligned}$$

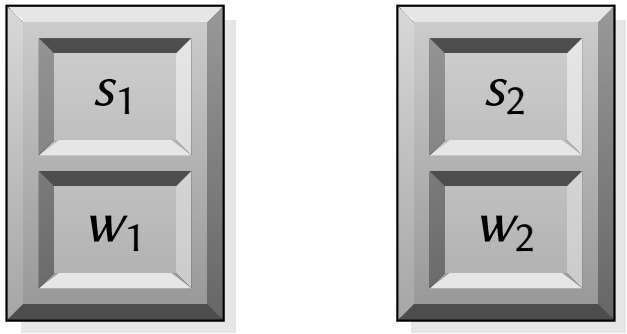
and

$$\begin{aligned} G_2: \quad & S_2 \to w_1 A_2 t_1 \mid w_2 A_2 t_2 \mid \ldots \mid w_n A_2 t_n \\ & A_2 \to \varepsilon \mid w_1 A_2 t_1 \mid w_2 A_2 t_2 \mid \ldots \mid w_n A_2 t_n. \end{aligned}$$

Fun fact: The PDAs for these grammars are deterministic, so we can invert them – so the complement of the languages of these two grammars are also CFLs.

Reducing from the PCP (4)

Suppose we made a sequence of tile 1 followed by tile 2:



So the top string is $s_1 s_2$. We can generate a related string in G_1 as follows:

$$S_1 \Rightarrow s_1 A_1 t_1 \Rightarrow s_1 s_2 A_1 t_2 t_1 \Rightarrow s_1 s_2 \varepsilon t_2 t_1 = s_1 s_2 t_2 t_1.$$

Note how the "$t_2 t_1$" lets us tell which tiles have been used. So we get the top string and which tiles have been used to get it.

Similarly, the second grammar generates

$$S_2 \Rightarrow w_1 A_2 t_1 \Rightarrow w_1 w_2 A_2 t_2 t_1 \Rightarrow w_1 w_2 \varepsilon t_2 t_1 = w_1 w_2 t_2 t_1.$$

Reducing from the PCP (4)

If the PCP instance has a match:

- There is a sequence of tiles with the same top and bottom strings.
- This sequence can be used to generate the same string in both grammars.
- So the grammars have a non-empty intersection.

If the PCP instance has no match:

- Any CFG strings generated from different tile sequences will have different suffixes (the t_is at the end of the string will be different).
- If we use the same tile sequence to generate a string from both grammars, the prefix (the non-t_i section) will be different.
- So the grammars will have no common strings — they have an empty intersection.

Reducing from the PCP (5)

So G_1 and G_2 generate a common string iff our PCP instance is a *yes*-instance.

> *If we can say that the PCP is undecidable, then so is the CFG intersection problem!*

A very similar reduction can be used to show that it's undecidable to determine if a CFG is ambiguous

> *Really, you just use the G_1 and G_2 above, but connect them with a rule:*

$$S \to S_1 | S_2.$$

But we use CFGs to define programming languages, and we use CFG parse trees of the code when we compile!

This is one reason we restrict the CFGs we use for programming languages. *And so, here's a place where decidability shows up in real life, too.*

Computability: A Look Back

This finishes our discussion of computability. The basic ideas we've covered are:

- Formalizing our notion of computation so that we can formally talk about what is computable.
- A specific formalization of computation: Turing machines and their configurations.
- An undecidable problem: HALT.
- Different levels of undecidability: recognizability and co-recognizability.
- Characterizations of recognizability: Recognizers, Certificates and Verifiers, and Enumerators.
- Using mapping reductions to get lots of results showing non-decidability / recognizability / co-recognizability.
- Embedding TMs into non-TM problems to get general undecidability results.